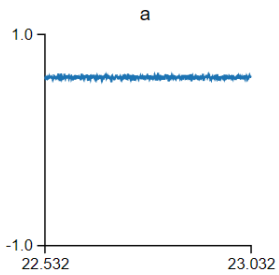
Neuromorphic Computing Week 3

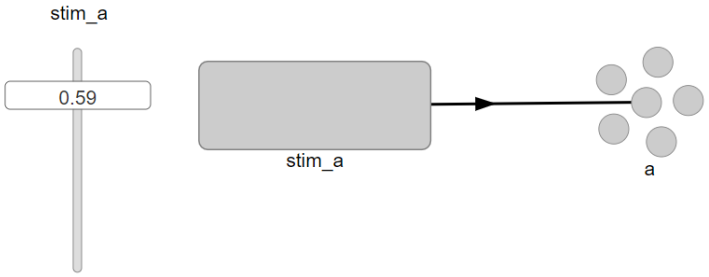
Contents

Exercise 1.1 2
Exercise 1.2 5
Exercise 1.3 7
Exercise 1.4 9
Exercise 1.5 11
Exercise 2.1 12
Exercise 2.2 14

Exercise 1.1

- The neurons are very accurate at representing the input.

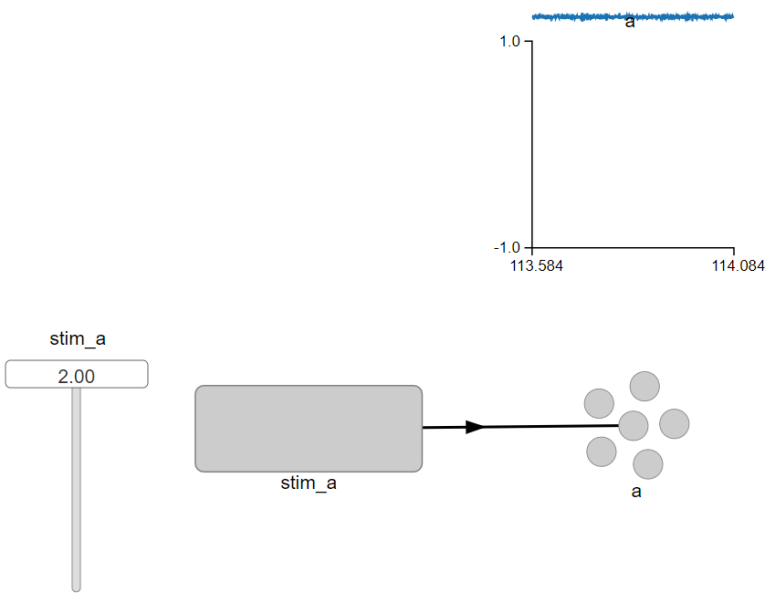




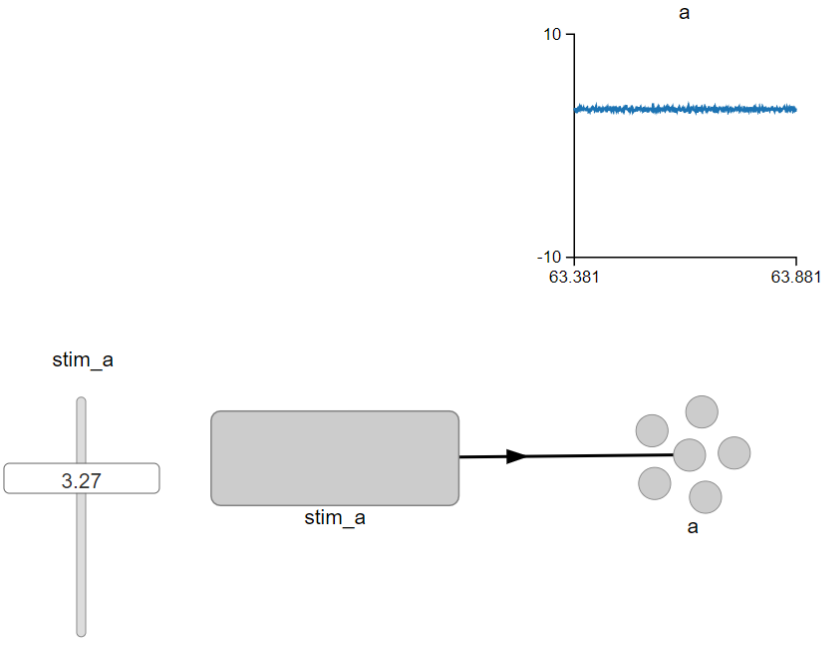

```
import nengo

model = nengo.Network()
with model:
    stim_a = nengo.Node(0)
    a = nengo.Ensemble(n_neurons=100, dimensions=1)
    nengo.Connection(stim_a, a)
```

- When the input values are of the range (-2,2), the neurons are not able to represent it well. The output line is out of the graph and it does not look like it is at 2, when input value is 2. It seems to be around 1.2.



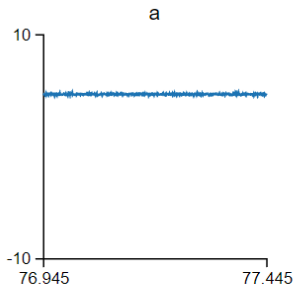
- Setting the radius of the Ensemble at 10 makes it good at representing values in the range of (-10, 10).
  The accuracy of the representation is decreased, but not by that much. It is still differentiable which value is put in, even though more noise is in the signal.

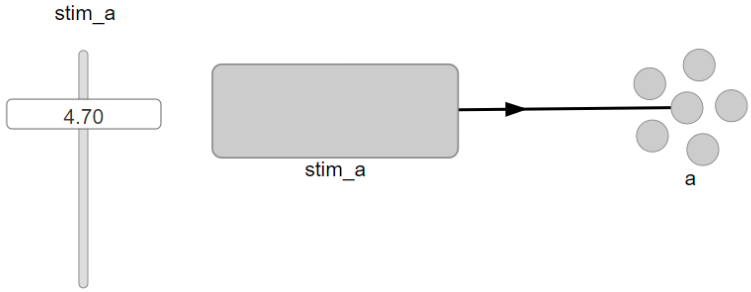



```
import nengo

model = nengo.Network()
with model:
    stim_a = nengo.Node(0)
    a = nengo.Ensemble(n_neurons=100, dimensions=1, radius = 10)
    nengo.Connection(stim_a, a)
```

- The output signal is less noisy, which increases the accuracy of the representation.



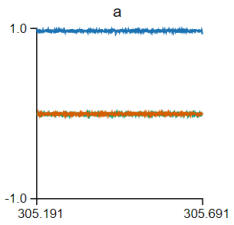


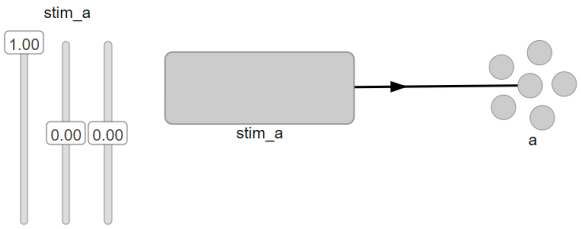

```
import nengo

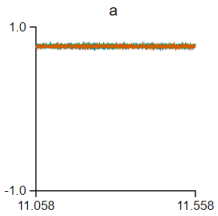
model = nengo.Network()
with model:
    stim_a = nengo.Node(0)
    a = nengo.Ensemble(n_neurons=500, dimensions=1, radius = 10)
    nengo.Connection(stim_a, a)
```

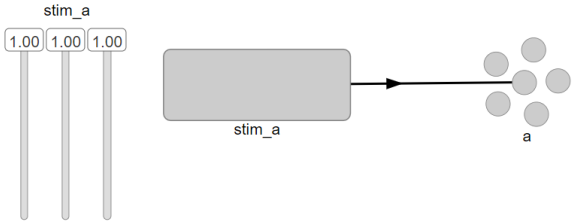
Exercise 1.2

- When other values are 0 it represents the value well, however, when at least one other value is higher than 0, the range of the output signal becomes more limited as seen below. As seen in the tutorial, when multidimensional values are represented, the radius of the output signal should be the square root of the input ranges added. So, it should be the square root of 3, which is around 1.7.





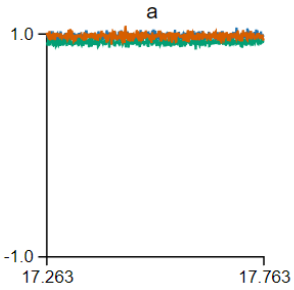


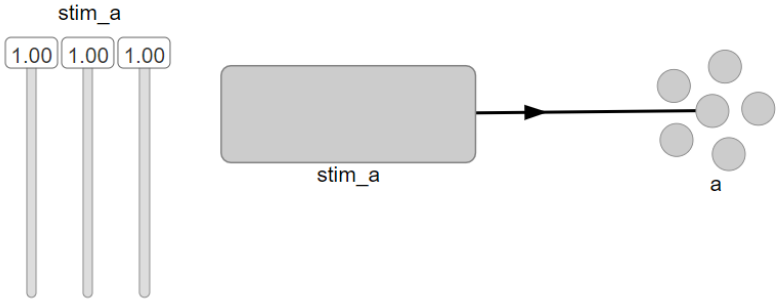



```
import nengo

model = nengo.Network()
with model:
    stim_a = nengo.Node([0,0,0])
    a = nengo.Ensemble(n_neurons=300, dimensions=3)
    nengo.Connection(stim_a, a)
```

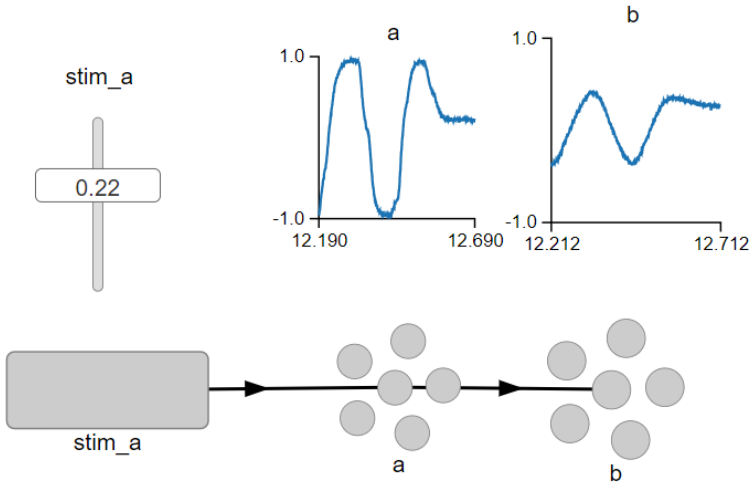
- Adjusting the radius to 1.7 gives a much better representation, as the rule that was mentioned above was followed.



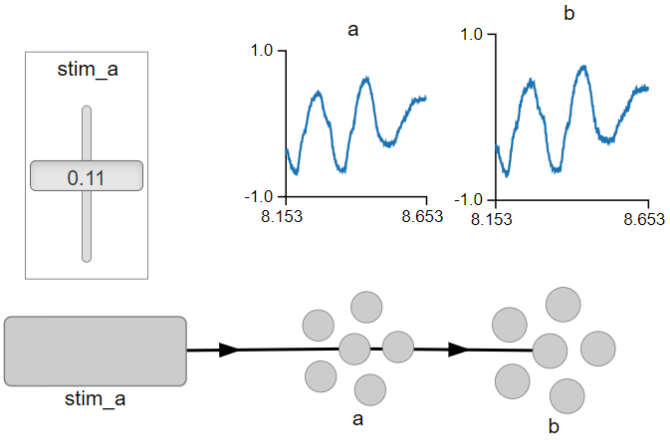


Exercise 1.3

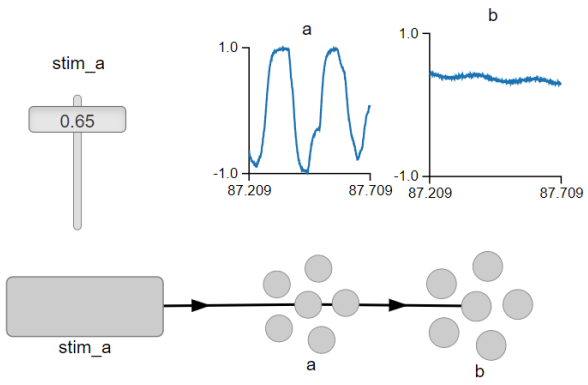
- When setting the synapse to 0.1, we see that fast changes in the first ensemble result in slower changes in the second ensemble output. This is seen in the picture below.



- Changing the synapse to 0.001 leads to almost the same input values.



- Changing the synapse to 1 leads to a very different output value:



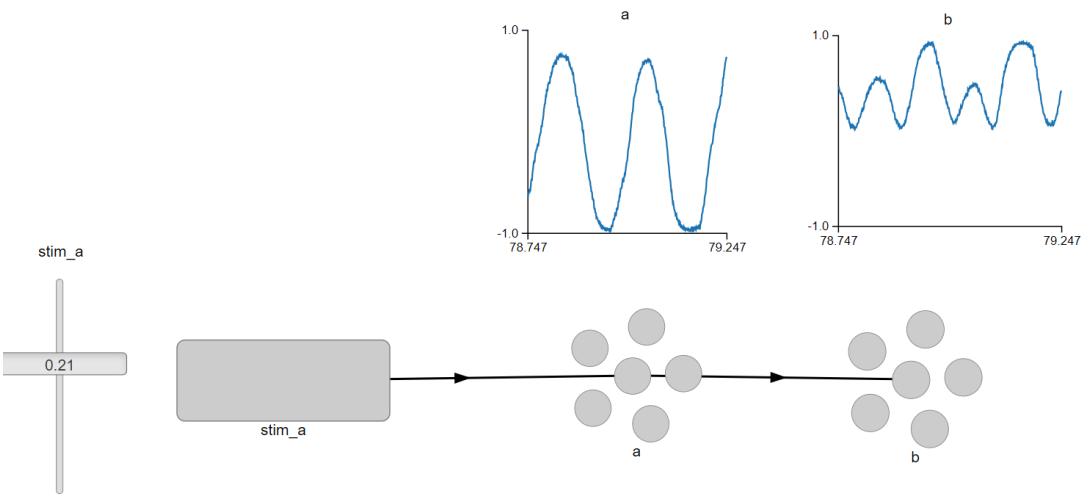

```
import nengo

model = nengo.Network()
with model:
    stim_a = nengo.Node(0)
    a = nengo.Ensemble(n_neurons=100, dimensions=1)
    b = nengo.Ensemble(n_neurons=100, dimensions=1)
    nengo.Connection(stim_a, a)
    nengo.Connection(a, b, synapse = 1)
```

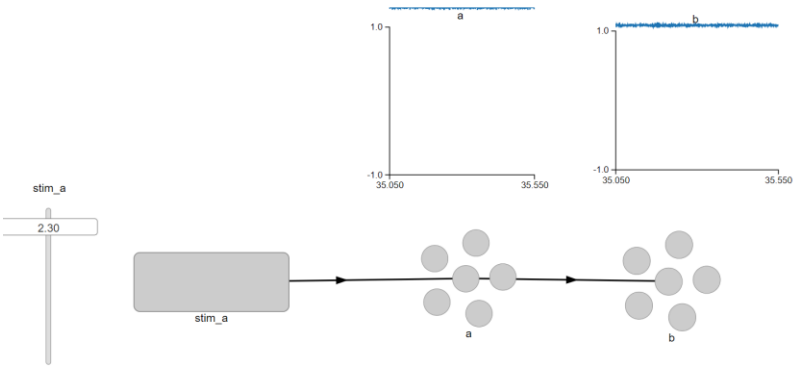
B is updated slow compared to A when increasing the synapse. A higher synapse value leads to less accuracy in representing the input ensemble.

Exercise 1.4

- Squaring the input value results in a very accurate output signal. Noise is very low and the output value is recognizable for input values in range of -1 and 1



However, when input value ranges are outside of -1 and 1, the signals are not represented well. A and B decoded graphs are out of range.



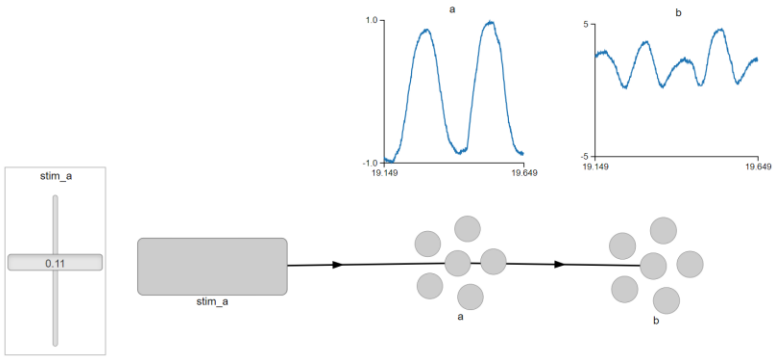

```
import nengo

model = nengo.Network()
with model:
    stim_a = nengo.Node(0)
    a = nengo.Ensemble(n_neurons=100, dimensions=1)
    b = nengo.Ensemble(n_neurons=100, dimensions=1)
    nengo.Connection(stim_a, a)

    def square(x):
        return x[0] * x[0]

    nengo.Connection(a, b, function = square)
```

- The result is rather accurate still, though it is noisier than before. The max output value is now 5, so the radius has been changed to represent the values.



More neurons should be added to the second Ensemble to accurately represent the output values. This is because the feature space is bigger than before and more neurons are needed to fill it. No neurons should be needed to add to Ensemble A, as that feature space stayed the same.

```
import nengo

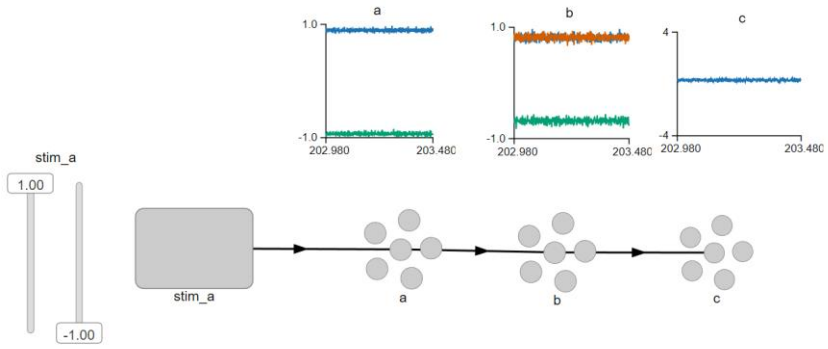
model = nengo.Network()
with model:
    stim_a = nengo.Node(0)
    a = nengo.Ensemble(n_neurons=100, dimensions=1)
    b = nengo.Ensemble(n_neurons=100, dimensions=1, radius = 5)
    nengo.Connection(stim_a, a)

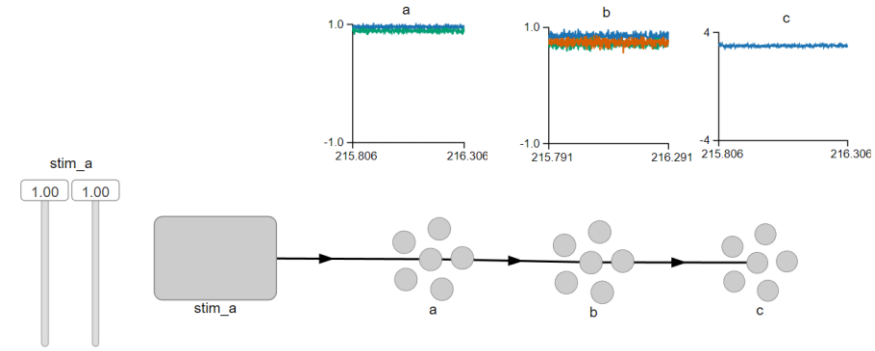
    def my_function(x):
        return 4 * x[0]**2 + x[0]**3


    nengo.Connection(a, b, function = my_function)
```

Exercise 1.5

- The system is computing (a+b)^2.
- The output of the last ensemble is: $a^2 + 2ab + b^2$
- The model is not that accurate, as it does not reach the top limits (4 and -4). This is because input values of both 1 or -1 are not represented well in b and c, even though radii have been adjusted.






```
import nengo

model = nengo.Network()
with model:
    stim_a = nengo.Node([0,0])
    a = nengo.Ensemble(n_neurons=100, dimensions=2, radius = 1.2)
    b = nengo.Ensemble(n_neurons=100, dimensions=3, radius = 1.7)
    c = nengo.Ensemble(n_neurons=100, dimensions=1, radius = 4)

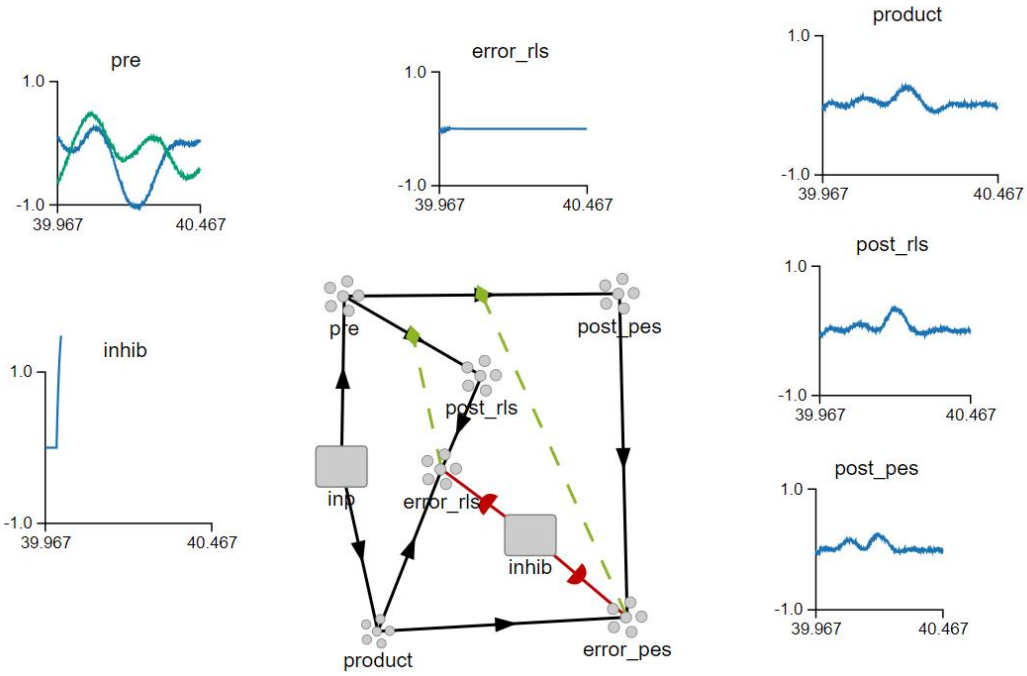
    def my_function(x):
        return  [x[0] * x[0], x[0]* x[1], x[1]*x[1]]

    def my_function2(x):
        return x[0] + 2 * x[1] + x[2]

    nengo.Connection(stim_a, a)
    nengo.Connection(a, b, function = my_function)
    nengo.Connection(b, c, function = my_function2)
```

Exercise 2.1

'Learning to compute a product' has been chosen as the model that will be benchmarked.





```python
import numpy as np
import matplotlib.pyplot as plt
import nengo
from nengo.processes import WhiteSignal

model = nengo.Network()
with model:
    # -- input and pre popluation
    inp = nengo.Node(WhiteSignal(60, high=5), size_out=2)
    pre = nengo.Ensemble(120, dimensions=2)
    nengo.Connection(inp, pre)

    # -- post populations
    post_pes = nengo.Ensemble(60, dimensions=1)
    post_rls = nengo.Ensemble(60, dimensions=1)

    # -- reference population, containing the actual product
    product = nengo.Ensemble(60, dimensions=1)
    nengo.Connection(inp, product, function=lambda x: x[0] * x[1], synapse=None)

    # -- error populations
    error_pes = nengo.Ensemble(60, dimensions=1)
    nengo.Connection(post_pes, error_pes)
    nengo.Connection(product, error_pes, transform=-1)
    error_rls = nengo.Ensemble(60, dimensions=1)
    nengo.Connection(post_rls, error_rls)
    nengo.Connection(product, error_rls, transform=-1)

    # -- learning connections
    conn_pes = nengo.Connection(
        pre,
        post_pes,
        function=lambda x: np.random.random(1),
        learning_rule_type=nengo.PES(),
    )
    nengo.Connection(error_pes, conn_pes.learning_rule)
    conn_rls = nengo.Connection(
        pre,
        post_rls,
        function=lambda x: np.random.random(1),
        learning_rule_type=nengo.RLS(),
    )
    nengo.Connection(error_rls, conn_rls.learning_rule)

    # -- inhibit errors after 40 seconds
    inhib = nengo.Node(lambda t: 2.0 if t > 40.0 else 0.0)
    nengo.Connection(inhib, error_pes.neurons, transform=[[-1]] * error_pes.n_neurons)
    nengo.Connection(inhib, error_rls.neurons, transform=[[-1]] * error_rls.n_neurons)

    # -- probes
    product_p = nengo.Probe(product, synapse=0.01)
    pre_p = nengo.Probe(pre, synapse=0.01)
    post_pes_p = nengo.Probe(post_pes, synapse=0.01)
    error_pes_p = nengo.Probe(error_pes, synapse=0.03)
    post_rls_p = nengo.Probe(post_rls, synapse=0.01)
    error_rls_p = nengo.Probe(error_rls, synapse=0.03)
```

Exercise 2.2

Parameters of interest:

- Pre population neuron count
- Input dimensionality size

```
import numpy as np
import matplotlib.pyplot as plt
!pip install nengo
import nengo
from nengo.processes import WhiteSignal
import timeit
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting nengo
  Downloading nengo-3.2.0-py3-none-any.whl (535 kB)
     |████████████████████████████████| 535 kB 6.8 MB/s
Requirement already satisfied: numpy>=1.19 in /usr/local/lib/python3.7/dist-packages (from nengo) (1.21.6)
Installing collected packages: nengo
Successfully installed nengo-3.2.0
```

```
start = timeit.default_timer()
model = nengo.Network()
with model:
    # -- input and pre popluation
    inp = nengo.Node(WhiteSignal(60, high=5), size_out=2)
    pre = nengo.Ensemble(30, dimensions=2)
    nengo.Connection(inp, pre)

    # -- post populations
    post_pes = nengo.Ensemble(60, dimensions=1)
    post_rls = nengo.Ensemble(60, dimensions=1)

    # -- reference population, containing the actual product
    product = nengo.Ensemble(60, dimensions=1)
    nengo.Connection(inp, product, function=lambda x: x[0] * x[1], synapse=None)

    # -- error populations
    error_pes = nengo.Ensemble(60, dimensions=1)
    nengo.Connection(post_pes, error_pes)
    nengo.Connection(product, error_pes, transform=-1)
    error_rls = nengo.Ensemble(60, dimensions=1)
    nengo.Connection(post_rls, error_rls)
    nengo.Connection(product, error_rls, transform=-1)

    # -- learning connections
    conn_pes = nengo.Connection(
        pre,
        post_pes,
        function=lambda x: np.random.random(1),
        learning_rule_type=nengo.PES(),
    )
    nengo.Connection(error_pes, conn_pes.learning_rule)
    conn_rls = nengo.Connection(
        pre,
        post_rls,
        function=lambda x: np.random.random(1),
        learning_rule_type=nengo.RLS(),
    )
    nengo.Connection(error_rls, conn_rls.learning_rule)

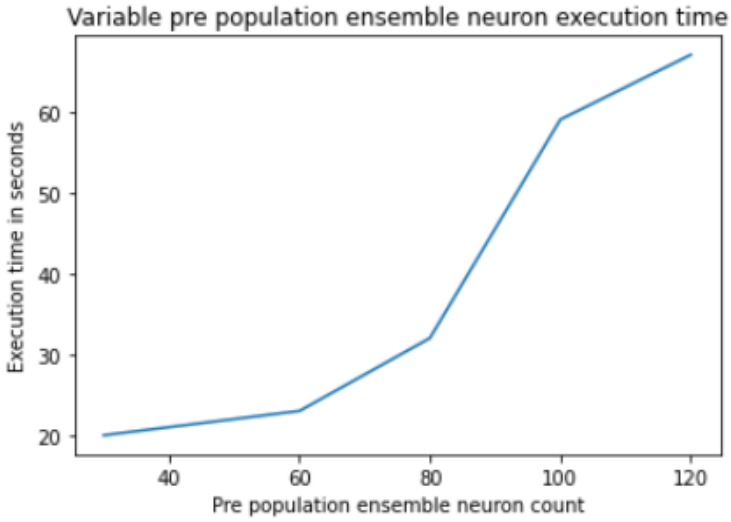
    # -- inhibit errors after 40 seconds
    inhib = nengo.Node(lambda t: 2.0 if t > 40.0 else 0.0)
    nengo.Connection(inhib, error_pes.neurons, transform=[[-1]] * error_pes.n_neurons)
    nengo.Connection(inhib, error_rls.neurons, transform=[[-1]] * error_rls.n_neurons)

    # -- probes
    product_p = nengo.Probe(product, synapse=0.01)
    pre_p = nengo.Probe(pre, synapse=0.01)
    post_pes_p = nengo.Probe(post_pes, synapse=0.01)
    error_pes_p = nengo.Probe(error_pes, synapse=0.03)
    post_rls_p = nengo.Probe(post_rls, synapse=0.01)
    error_rls_p = nengo.Probe(error_rls, synapse=0.03)

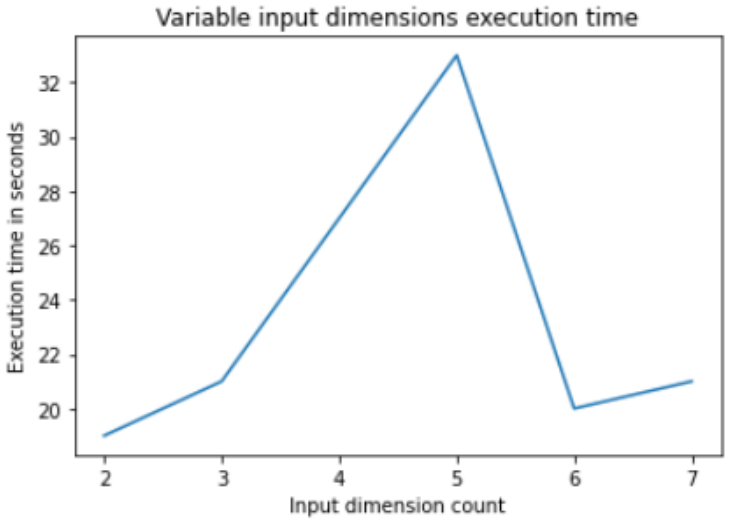
with nengo.Simulator(model) as sim:
    sim.run(40)

end = timeit.default_timer()
print(end - start)
```

```
prepop = [30, 60, 80, 100, 120]
prepop_time = [20, 23, 32, 59, 67]
plt.plot(prepop, prepop_time)
plt.xlabel("Pre population ensemble neuron count")
plt.ylabel("Execution time in seconds")
plt.title("Variable pre population ensemble neuron execution time")
plt.show()
```




```
input_dim = [2, 3, 4, 5, 6, 7]
input_dim_time = [19, 21, 27, 33, 20, 21]
plt.plot(input_dim, input_dim_time)
plt.xlabel("Input dimension count")
plt.ylabel("Execution time in seconds")
plt.title("Variable input dimensions execution time")
plt.show()
```



Code ran for 40 timesteps, but the execution time was variable, as seen in the plots above. Increasing the pre population neuron count increases the execution time almost exponentially, as expected. Increasing the neurons present in computation should increase the execution time, as more neurons are learning. The neuron population of “Pre” was variable in this experiment.

The input dimensionality was variable in this experiment as well, though something weird happened here. Execution time was expected to keep rising, as the dimension kept increasing. I thought this was logical, as the feature space is increasing at a high rate. This was true up to 5 dimensions, though the 6th dimension reduced execution time to a lower level than 3 dimensions. A reason for this might be that the program does not handle high dimensionality well and some computation is thrown away. This is pure speculation, as I do not know the cause for this. The dimensionality of “inp” and “pre” were changed during this experiment.

To conclude, the parameters of interest do affect simulation time.